Advanced Java

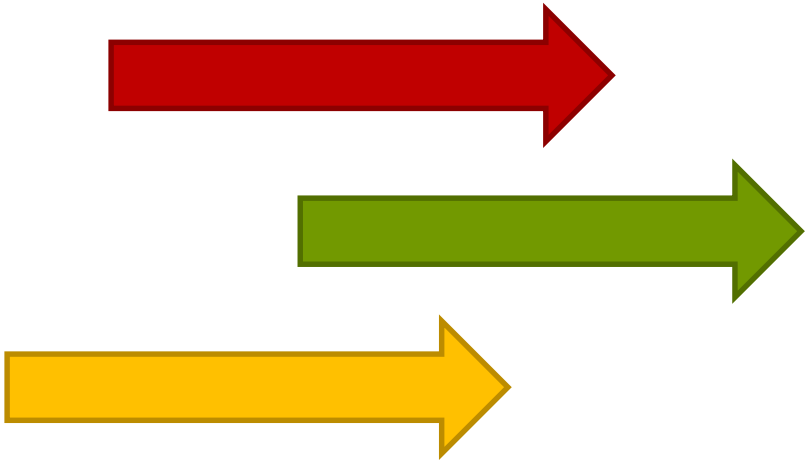
Concurrent programming

Topics

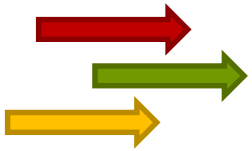
- Introduction to concurrency
- Execution model
- Memory model implications
- Practical basics
- Design concerns
- Multithreading support
- Building blocks
- Concurrent troubles

About concurrency

- Concurrency and parallelism
- Terminology
- Motivations and dangers



Concurrency × parallelism



- Concurrent actions
  - Existing within overlapping time frames
- Parallel actions
  - Being executed in the same time (and thus being concurrent)

↳ Parallelism is just one way of concurrency

- However, probably the most complex case, bringing its own problems and challenges

Why to care at all?

- Performance gains
  - Due to physical limits for increasing hardware speed, parallelism must be exploited to get more computation power
- Support for natural concurrency
  - Users require multiple operations in progress at the same time (e.g., UI should be able to react independently on other proceeding actions)
- More powerful and easier-to-use techniques
  - E.g., threads, coroutines, iterators etc.

A few definitions

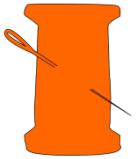
- Concurrent programming needs & brings appropriate abstractions

- Thread (of execution)
  - The smallest sequence of instructions that can be managed independently by a (system) scheduler and executed independently as well
  - It is an active entity, unlike a process, it does actually “run”

- Process
  - An instance of a program that is being executed
  - A collection of various resources (including memory) and a container for threads
  - It’s a passive entity, a process without threads would do nothing

- Shared resource
  - A resource (an entity required for a thread to run) accessible to multiple processes, or rather to multiple threads

Why to bother with threads?



- Multithreading is built in the very core of Java and its runtime environment
- Knowledge of multithreading and concurrency is implicitly assumed
- Good coding and design practices must count with that
- Surprisingly, a lot of developers lack deep understanding of the essentials… which leads sometimes to catastrophic failures

Pitfalls and dangers

- Concurrency threatens (naïve) consistency
  - Care and some guarantees are requires to achieve the desired (formal) consistency
  - Programmers have to understand the essentials very well
- Concurrent actions result in unpredictable outcomes
  - However, if desired consistency is kept, the outcomes can still be valid and reasonable
  - Anyway, excluding the possibility of invalid outcomes is a challenge
- Bugs in concurrent code are very difficult to find
  - Invalid outcomes can appear rarely or depend on particular platforms (even in Java)

Discovering the parallel reality

- Heading to the core of parallelism
- Real hardware considerations

Sharing a resource (or a state)

Thread A

```
int amount = …
balance = balance + amount;
```

```
int balance;
```

A shared piece of memory

Thread B

```
int amount = …
balance = balance + amount;
```

- When sharing a resource…
  …multiple threads may access it
- When touching it concurrently
  …a conflict may occur
- When a conflict occurs…
  …the outcome is indeterminate
- Preventing concurrent access prevents the conflict and prevents threads from ruining the balance
- That's why locks were invented
  - To prevent unwanted access ;-)
  - To make an action appear atomic

About concurrent programming

"Perhaps surprisingly, concurrent programming isn't so much about threads or locks, any more than civil engineering is about rivets and I-beams.

Writing thread-safe code is, at its core, about managing access to state, and in particular to shared, mutable state."

Brian Goetz

- Alright, this **is** important, but there is something more yet
  - We'll have to deal with threads and locks for a while to find it out

Let's write a lock…
A very, very simple lock
Can you?

Ehm…

A very simple lock

```
Lock::lock

int tid = currentThreadId();
while (!OWNER.compareAndSet(this, 0, tid)) {}
```

```
int owner;
```

A shared piece of memory – the instance of `Lock`

```
Lock::unlock

int tid = currentThreadId();
if (!OWNER.compareAndSet(this, tid, 0)) {
    throw new Error(…);
}
```

The `compareAndSet` operation must compare a value in a variable and set it to a given value **atomically**

That's cheating a bit, I admit… but:

- such an operation is supported by the hardware usually (so called CAS instructions)
- there are solutions that can work without that (Peterson's algorithm, Dekker's algorithm), but still with some assumptions about the hardware

Sharing… revisited

Thread A

```
int amount = …
L.lock();
balance = balance + amount;
L.unlock();
```

```
int balance;
Lock L;
```

A shared piece of memory

Thread B

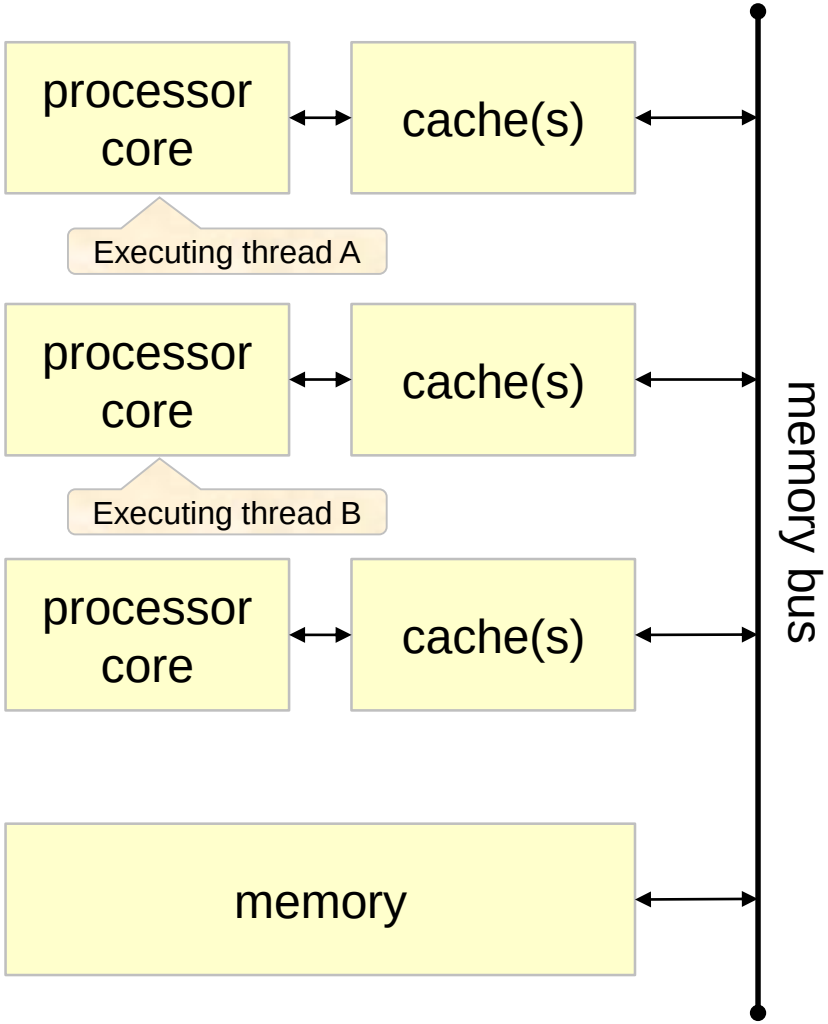
```
int amount = …
L.lock();
balance = balance + amount;
L.unlock();
```

If the lock works, the conflict is avoided…

…so we are safe, right?

Right?

About memory architecture



We would certainly be safe…

- if there were a single CPU core
- or if all (?) memory operations were synchronous
- and the CPU(s) were somewhat simple (which needs a lengthy explanation)

Otherwise thread A may **see** a different value at the same memory address than thread B (because of caching… mostly)

↳ We have to consider memory **visibility** for particular threads

Having all memory access synchronous is too harsh and inefficient, but it might be sufficient to synchronize just something…

About real hardware

- Hardware must face difficult compromises
  - Memory transactions depend on cache lines and bus width
  - Memory transactions become even more challenging with growing number of attached components… like cores
  - Word width and bus width may differ (which may be one of the reasons for word tearing)
  - No particular outcome guaranteed with data races
- Hardware attempts peculiar things
  - Instructions may be executed out of the original order, in parallel or even speculatively
- There are some physical limits
  - How far light reaches within a tick of a CPU clock?

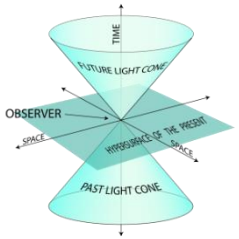
Time perspective

Access	Size	Latency [ns]	Latency scaled for humans
Register	1–32 B	0.3	1 s
L1 cache	~ 32 KB	1.2	4 s
L2 cache	~ 256 KB	3	10 s
L3 cache	~ 8 MB	12	40 s
Local RAM		60	3 min 20 s
Remote RAM		100	5 min 33 s
SSD I/O		~ 50 000 (50 µs)	46 h 17 min
HDD I/O (magnetic)		~ 4 000 000 (4 ms)	~ 155 days

Execution model

- Concurrency basic rules
- Defining a model
- Trade-offs

Concurrency basic rules



- **All actions must be considered with the respect to their mutual order**, i.e., with the respect to their executors and the observers
- **The visibility of the outcomes is essential**

↳ **We need to ensure a well-defined ordering of all actions and the visibility of their outcomes**

"What *happens before* this action and who can see the results?"

Execution model in general

- Syntax of a programming language is not interesting for the time being: it tells how to write the source, but not how the code executes
- Semantics of a programming language is now important: it is described via the semantics of the abstract machine executing the source program
- Significant part of the machine specifications are the rules concerning how memory storage behaves: the memory model
- A memory model must answer a simple question: **what values can a particular *read* in a program return?**

Execution model and Java

- Java is an imperative language (the code implies some kind of direct execution)
- Java specification sticks to natural code reading (top-down, left-to-right)

➪ Execution model seems (locally) sequential

✓ Java is very programmer-friendly and covers infamous gaps known, e.g., from C/C++

? However, misunderstanding can come easily, e.g., as the result of a simplistic interpretation

Memory model trade-offs

- Memory model must consider:
  - How difficult is using the language
  - How difficult is implementing the language
  - How difficult is building appropriate hardware

- Java Memory Model deserves highest credit
  - Providing quite complete formal specification
  - Taking into account various platforms, while retaining all the portability
  - Still sane enough, so that the programmers would not go crazy

Quiz

Let’s have a look…

Correct or incorrect?

```
public final class Box<T> implements Supplier<T> {

    private T value;

    public Box() {
        // Default constructor
    }

    public Box(T argument) {
        value = argument;
    }

    public T get() {
        return value;
    }

    public synchronized void set(T argument) {
        value = argument;
    }
}
```

And what about now?

```
public final class Box<T> implements Supplier<T> {

    private T value;

    public Box() {
        // Default constructor
    }

    public Box(T argument) {
        value = argument;
    }

    public synchronized T get() {
        return value;
    }

    public synchronized void set(T argument) {
        value = argument;
    }
}
```

Wait. Is it really necessary? It was **synchronized**, *wasn't it?*

What should be better here?

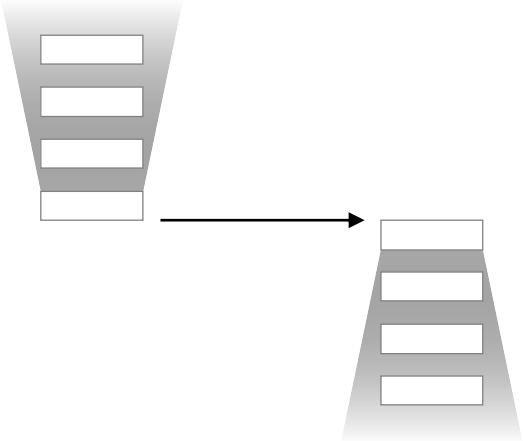
```
public final class DoublingSupplier
    implements IntSupplier {

    private int value;

    public DoublingSupplier(int parameter) {
        value = parameter * 2;
    }

    public int getAsInt() {
        return value;
    }
}
```

This field should have been `final`.

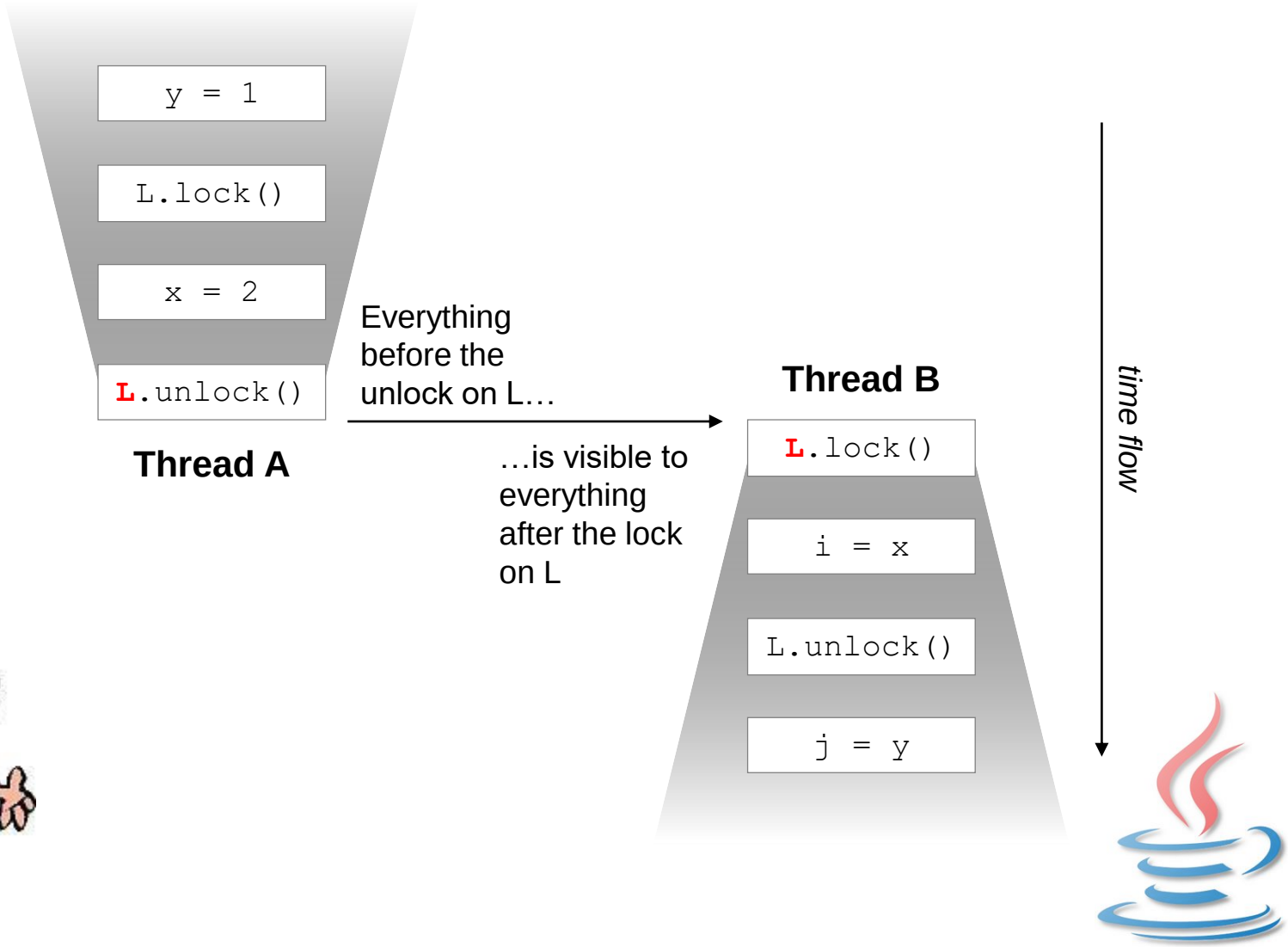
Come on. Writing `final` is annoying. It's useless anyway.

Java Memory Model

- Definition and rules
- Consequences
- Techniques



Java Memory Model in a picture



Java Memory Model in 8 minutes

- Java Memory Model (JMM) defines a **partial order** called *happens-before*:

  "If action A *happens before* action B, then action B can see the results of action A (whether or not A and B occur in different threads)"

- The (basic) rules for the "happens-before" order:
  - **Program order rule**: each action in a thread happens before every action in that thread that comes later in the program order
  - **Lock rule**: an unlock of a lock happens before every subsequent lock on that same lock (this applies on library locks as well as on intrinsic locks)
  - **Volatile variable rule**: a write to a volatile field happens before every subsequent read of that same field (this applies on atomic variables too)
  - **Thread start rule**: a request to start a thread happens before every action in the started thread
  - **Thread termination rule**: any action in a thread happens before any other thread detects that thread has terminated
  - **Interruption rule**: requesting an interruption on a thread happens before the interrupted thread detects the interruption request
  - **Finalizer rule**: the end of an object's constructor execution happens before the start of the finalizer for that object

JMM formally: the big picture

- JMM describes the allowed program outcomes by describing what *executions* are conforming
- It does so by introducing *actions*, and *orders* over actions, and *consistency rules* that govern what actions, orders and constraints constitute a valid execution
- **If a program result can be explained by some valid execution, then this result is allowed under JMM**

JMM formally: action orders

- **Program order** (PO): defines a total order of actions within each thread

- **Synchronization order** (SO): defines a total order over synchronization actions (explicitly listed in the specification)
  - **SO consistency**: all reads that come later in SO see the last write to that location
  - **SO-PO consistency**: the order of synchronization actions in SO within a single thread is consistent with PO in that thread
  - ↳ SO consistency and SO-PO consistency mean that in all conforming executions synchronized actions appear sequentially consistent

- **Synchronized-with order** (SW): a partial suborder of SO that pairs specific synchronization actions that "see" each other
  - This order serves as the connection bridge between different threads

JMM formally: happens-before

- **Happens-before order** (HB): the transitive closure of the union of PO and SW
  - A partial order, and only relates **some** actions, **not** every pair of actions
  - Able to relate the actions that are not synchronization actions, which allows us to cover all important actions with ordering guarantees
  - **HB consistency**: every read can see either the latest write in the happens-before order (notice the symmetry with SO here), or any other write that is not ordered by HB (this allows races)

JMM formally: additional rules

- **Causality rules**: additional verifications on otherwise conforming executions, to rule out causality loops

- **Final field rules**: providing additional constraints imposed by `final` fields, e.g., additional happens-before relationship between `final` field stores and their associated reads

Benefits of Java Memory Model

- Java Memory Model implies directly:
  - Locks provide both atomicity and visibility
  - Volatile variables provide just the visibility
  - Atomic variables (see later) provide the visibility and limited atomicity

- "Piggybacking" technique
  - The strength of "happens-before" allows to piggyback sometimes on the visibility guarantees that are implied by the existing "happens-before" enforcement
  - This technique is quite fragile, but can gain some performance and allows, e.g., safe using of various thread-safe collections in a natural way as a side-effect

- Reasonable price for achieving thread safety measured by the complexity and the imposed performance penalty

Stale data

- If a thread reads data from a memory place without ensuring the proper visibility guarantee, it may load *stale data* (the data which are not fresh because of a change performed by another thread in the memory place)

- What are the consequences?
  - Usually bad enough – it is the common cause of race conditions and other hazards… stale data can spoil anything

? But… there is something to know anyway: "out-of-thin-air" safety
  - ➢ This kind of safety tells that stale data can only contain the data written by a thread in the past (although not telling which thread and how old the data might be – you just know the data are not random)

  ☠ It does **not** apply on `double` and `long` values (see JVM internals)

  ± It applies on all other values (including references, but not objects!)

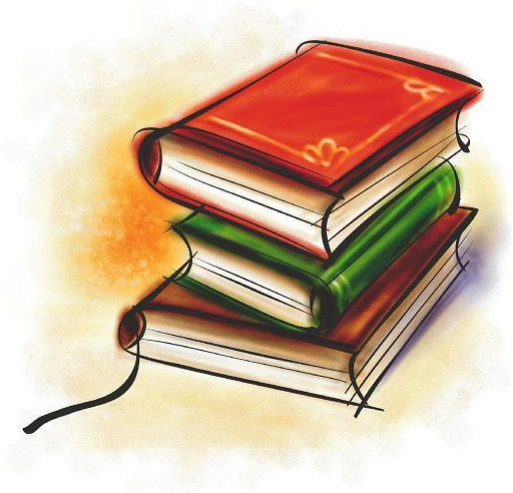
  ± This kind of safety has a use, e.g., for hash code caching technique

Often mistakes

- Coding based on anecdotal observations of what language runtimes are doing today
- Too harsh mental shortcuts, e.g., imaging barriers instead of using actual JMM guarantees
- Mixing happens-before order and physical execution order
- Using `volatile` solves all problems
- Java is portable, so when it runs correctly on my platform…

Concurrency in theory

- Safe publication, immutability
- Confinement
- Sharing policies



State management

"Writing thread-safe code is, at its core, about managing access to state, and in particular to shared, mutable state."

Brian Goetz

- We have to care about state and how it can be accessed
  - Stale data can spoil everything, out-of-thin-air safety does not save us in general
- Hence we have to answer following questions
  - When some state should or should not be visible?
  - How to achieve the conditions for safe access?
  - How to prevent unsafe access?
  - Which access policies can we actually declare?
- We have to revive the concept of ownership

Escape… and publication

- **Publishing** an object means making it available to code outside of the object's current scope
  - Publishing an object may indirectly publish others
  - Publishing can compromise encapsulation, break invariants… (e.g., due to leaking references to an object's internals)

- When an object is published when it should not have been, it is said to have **escaped**
  - It brings many hazards: abusing or misusing the object, accessing an incomplete instance (if the object escapes during its construction), viewing an inconsistent state of the object (publishing without considering memory visibility) etc.

- Never allow an object to escape, always ensure safe publication

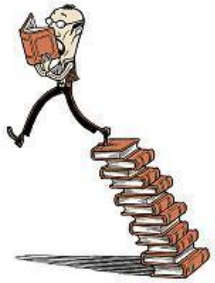
Intermezzo: immutability

- A truly **immutable** instance has just one state which can't be changed
- An **effectively immutable** instance has just one externally visible state which can't be changed

- Effective immutability is weaker
  - An effectively immutable class/instance needs more care to be really safe

- Immutability in general will be discussed later

Safe publication: considerations



Initialization safety: `final` field semantics

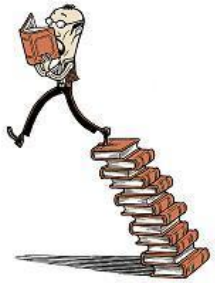
- **"A thread that can only see a reference to an object after that object has been completely initialized is guaranteed to see the correctly initialized values for that object's `final` fields"**
- This is another huge point in favor of (true) immutability and a help for other objects too (especially for achieving at least the effective immutability)

I thought that `final` was just...

What to watch carefully?

- ☠ Construction time: leaking incomplete instance
- ❗ Fields: inspect what is loaded and stored
- ❗ Return values: the most common way for an escape
- ❗ Method arguments: often not realized that they accept values
- ❗ Inner (non-static) class instances: contain the implicit reference to the outer instance

Safe publication: techniques



What is safe then?

A properly constructed object can be published by…

- Initializing the object reference in a `static` initializer
- Storing a reference to it into a `volatile` field or an atomic reference instance (e.g., `AtomicReference`)
- Storing a reference to it into a `final` field of a properly constructed object
- Storing a reference to it into a properly synchronized field
  - Either using explicit locks or using some form of piggybacking
  - Various thread-safe library collections can be included here

↳ Then reading from such a reference source provides a safely published object

Safe publication: summary

✓ Immutable objects can be published through any mechanism

± Effectively immutable objects must be safely published

😐 Mutable objects must be safely published and must be either thread-safe or guarded (see later)

❗ Always document the publishing requirement and the policy how to deal with the instances – it's a part of the contract

What?! Documentation?

Confinement: overview

Confining some data ~ preventing the data from sharing

- When an object is confined to a thread, its usage becomes thread-safe even if the confined object itself is not

- ✓ No sharing, no synchronization → this is the cheapest way
- ✓ It allows to use safely the code which is not thread-safe
- ✓ It can help to avoid deadlocks and overwhelming complexity
- ± It is just the matter of design and implementation
  - The compiler is not helpful at all

- Used very often, in many cases implicitly or “by the way”

Confinement: techniques

Technique	Description
Confining to dedicated privileged threads	Original and typical use, common for GUI frameworks, database connections…
Ad-hoc thread confinement	Delegating the execution of some single-threaded code to a single thread while sharing the data is avoided during the execution Correct memory visibility is required!
Stack confinement	The data are reachable through the local variables only → confined intrinsically
Instance confinement	Encapsulating a thread-unsafe instance to provide thread-safe access to it
Thread locals	Every thread can use the same “global” way to access a private copy of some data

Common sharing policies

- Thread confinement
  - A thread-confined object is owned exclusively by a thread
  - Only the owning thread can access the object
- Shared read-only
  - A shared read-only object can be accessed by multiple threads without additional synchronization, but can’t be modified by any thread
- Shared thread-safe
  - A thread-safe object performs synchronization internally
  - Multiple threads can freely access it through its public interface without further synchronization
- Guarded
  - A guarded object can be accessed only with a specific lock held
  - Guarded objects include those that are encapsulated within other thread-safe objects and published objects that are known to be guarded

Ownership

- When no garbage collector exists, the concept of ownership is much clearer: there is somebody responsible for destroying the instance
- But the concept of ownership remains valid even when a garbage collector takes this duty – somebody still may have privileges that the others lack (or should lack)

↳ This becomes apparent when considering thread safety – e.g., confinement manifests some kind of object ownership

➢ Ownership options: exclusive ownership, shared ownership, split ownership, ownership passing

Summary

What you should remember

Conclusion

- Actions' mutual order and visibility of their effects are essential
  - The order must be always proved using the memory model's rules
- Basic rules for *happens-before*
  - Especially "lock" and "volatile" rules
- Limit mutability, strive for true immutability – use `final`
  - Highly recommended, little known, frequently neglected
  - Limiting mutability is always a good idea anyway
- Always consider confinement
  - Often the cheapest and least complicated option
- Never allow an object to escape, publish safely + document
  - Watch out what becomes available outside an instance
  - Consider object ownership

Concurrency in practice

- Basic concurrency support
- Locks and atomic variables
- Common idioms

Concurrency support… so far

- Field types: `volatile` and `final`
  - Special semantics concerning memory visibility; `volatile` rules apply to atomic variables as well
- Locks: referred in Java Memory Model rules
  - Based on the monitor abstraction
  - Intrinsic: built-in, every object can be used as a lock when applying keyword `synchronized`
  - Explicit: implemented as classes in the runtime library
- Representation of a thread: class `Thread`

Questionnaire

- Have you experienced the situation when your code had to wait for something?
- Have you experienced the situation when your code was waiting for something and you needed/wanted to interrupt the waiting?
- In your opinion, how a blocking method shall be interrupted?
- In your opinion, what a blocking method should do when interrupted?
- In your opinion, how a blocking method should indicate an interruption?
- Have you ever met `InterruptedException`?
- How do you handle `InterruptedException`?

Blocking & interruptible methods

- ❓ A thread can't be forced to do something, it can't be even killed (except for terminating the whole JVM) – there are good reasons for that

- ➢ Java/JVM offers a cooperative mechanism for interruptions: an interruption is a way to **ask** a thread to 'stop' (something) by adjusting the thread's interruption state (see `Thread::interrupt` & friends)

- ➢ Blocking methods (usually) check (or should check) the interruption state and throw `InterruptedException` when the thread has been interrupted

- ✓ It is the most sensible handling in the general case, usual especially for library code and reusable components

Handling interruptions

- When calling such a blocking method, you have to deal with the exception

- Propagate the exception further
  - This way forces the calling method to declare `throws InterruptedException` which is not always possible (e.g., within `Runnable::run`)

- Swallow the exception and restore the interruption state of the current thread:

```
try {
    …
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

Intrinsic locks

- JVM built-in support (bytecode instructions `monitorenter` and `monitorexit`)
- Core supportive methods on the `Object` class: `notify`, `notifyAll`, `wait`
- Java language uses the **`synchronized`** keyword for emitting the appropriate JVM instructions

✓ Efficient and JVM may optimize it in run time

✓ Possible to detect deadlocks with tools easily

✗ Somewhat limited and not so efficient in more complex scenarios

Explicit locks: about

- Implemented in the runtime library using existing support in JVM
- Their capabilities may vary (depending on the implementation) and it is possible to make an own implementation
- See `java.util.concurrent.locks` package for the standard offer

✓ Much more powerful

✗ Not so easy to use usually

✗ Not so efficient (at least in the simple cases)

✗ Limited tool support (JVM optimization, deadlock detection etc.)

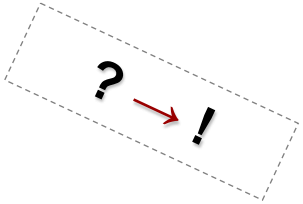
Explicit locks: explained

- Explicit locks: `Lock` (since Java 5)
  - Many new features over intrinsic locks (see the interface)

- Pros and cons of the interface
  - ✓ Timed and polled acquisition
  - ✓ Interruptible acquisition
  - ✓ Non-structured locking possible
  - ✓ Improved condition variable support (see later)
  - ✗ More difficult for JVM to optimize them or list them in thread dumps (no stack-frame relationship, harder diagnostics)
  - ☠ Easy to forget to unlock it: **always** unlock in `finally`

- The implementation further offers:
  - ✓ Read-write locking (see `ReentrantReadWriteLock`)
  - ✓ The **possibility** to use fair locking (at some cost!)
    - Both `ReentrantLock` and `ReentrantReadWriteLock`

Condition variables: theory

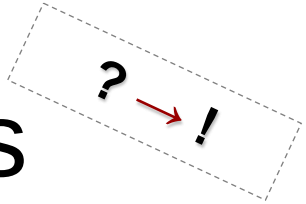


- State-based preconditions → state-dependent operations
  - In a concurrent environment it makes sense to wait for a precondition to become true, i.e., let the operation block (see, e.g., `BlockingQueue`)

- Structure of a blocking state-dependent operation:

```
? blockingAction() throws InterruptedException {
    acquire lock on object state
    while (precondition does not hold) {
        release lock
        wait until precondition might hold
        optionally fail if interrupted or timeout expires
        reacquire lock
    }

    perform action
    release lock
}
```

Condition variables: possibilities

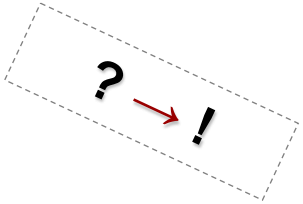


- Condition queues – how to deal with conditions variables
  - Implicit and simple: provided by intrinsic locks (see `Object`'s `wait`, `notify` and `notifyAll` methods)
  - Explicit and richer: provided by explicit locks (see `Condition` interface)

- Choosing the proper way to support conditions
  - ✗ Never ever wait in a loop with `sleep`: it's a sin
  - Use implicit condition queues when…
    - You don't need any advanced features
    - You don't have an explicit lock already
    - You have a "simple single" predicate

- Both implicit and explicit condition queues have similar rules and use, they follow the same abstraction after all

Condition variables: practice



- The condition predicate, a lock and a condition queue are associated
  - Every waiting in a condition queue is implicitly bound to the predicate
  - The association is not directly visible → document it

- Waking up does not mean the condition is (still) true
  - Another waiter could just steal the chance, or a spurious wake-up could occur

- Missed signals: some of the protocol rules was violated
  - Ensure the predicates are guarded with the appropriate lock and hold the lock when going to wait on the predicate
  - Always have a condition for waiting and test it before and after waiting
  - Always invoke waiting in a loop (helps with the above too)

- Missing notifications: was something forgotten?
  - Ensure the waiters will be woken up when the condition becomes true
  - When unsure, initiate a wake-up: e.g., when `InterruptedException` occurs
  - You may and should notify a single waiter iff both following statements hold true:
    - Uniform waiters: only one condition is associated with the condition queue, and each thread executes the same logic upon waking up
    - One-in, one-out: a notification enables at most one thread to proceed

Atomic variables

Classes in `java.util.concurrent.atomic`

- They have volatile read/write semantics (i.e., form *happens-before* relation as `volatile` does)
- Extra compound operations
  - Get-and-set and compare-and-set operations
  - Change-and-get for `AtomicInteger` and `AtomicLong`
  - Marking and stamping for `Atomic*Reference`

➢ The atomic variables are important means for light-weight synchronization and non-blocking techniques

Variable handles

Java 9 introduced variable handles (“var handles”)

- An add-on of method handles introduced in Java 7 to provide a standard API for some of the powers possessed by `sun.misc.Unsafe`

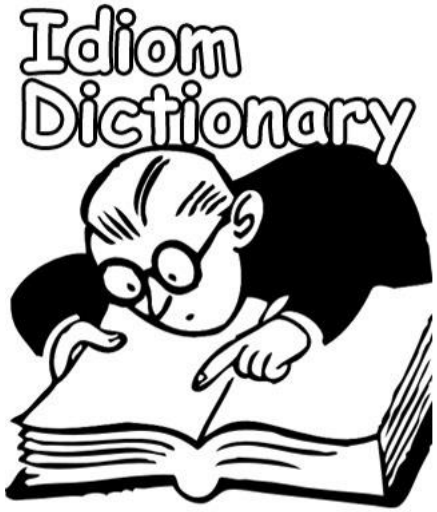
✓ Allow fine-tuning memory operations (all JMM model memory operation levels available) and explicit fencing

✓ An optimized alternative to atomic variables

❗ But requiring a deep understanding of JMM

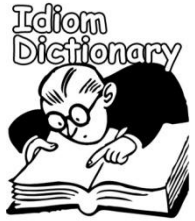
Idioms

- Double-checked locking
- Lazy initialization holder
- Final wrapper



Warning!

Double-checked locking

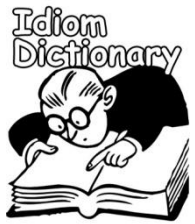


- Prior Java 5: broken (insufficient `volatile` support)
- Still often implemented wrong or misunderstood; this is a correct code snippet applied on a singleton (the original common use case):

```
public final class ResourceFactory {

    private static volatile Resource INSTANCE;

    public static Resource getInstance() {
        Resource result = INSTANCE;

        if (result == null) {
            synchronized (ResourceFactory.class) {
                result = INSTANCE;
                if (result == null) {
                    result = new Resource(…);
                    …
                    INSTANCE = result;
                }
            }
        }

        return result;
    }
}
```

Warning!

Double-checked locking

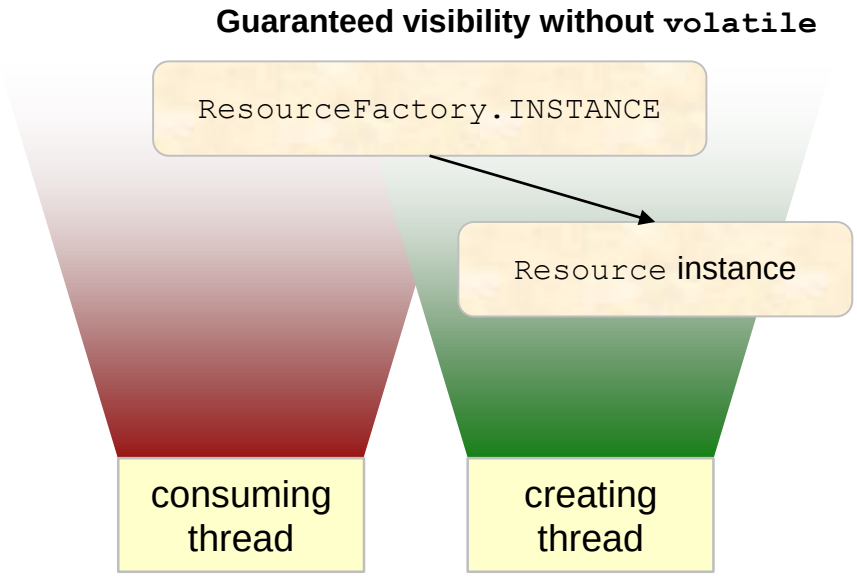


Why `volatile` is necessary?

- Imagine `Resource` implementation like this:

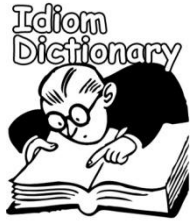
```
public final class Resource {

    …

    private String data;

    public Resource(…) {
        …
        data = …
        …
    }

    public String getData() {
        return data;
    }
}
```

!



- The instance is **effective** immutable **only**
  - Such an instance requires already safe publication which does not occur if `INSTANCE` in the double-locking idiom has not `volatile` semantics

Lazy initialization holder



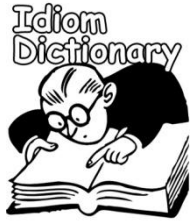
- Double-checked locking was broken before and implemented often incorrectly → for singletons (the common use case) a safer idiom was developed

```
public final class ResourceFactory {

    private static class ResourceHolder {
        public static final Resource RESOURCE
            = new Resource();
    }

    public static Resource getResource() {
        return ResourceHolder.RESOURCE;
    }
}
```

- This idiom uses the class-loading & initialization process to ensure the safe publication and synchronization of the resource construction

😐 Unfortunately, it is usable for singletons only as it relies on the class instantiation (and `Class` instances are singletons)

Final wrapper

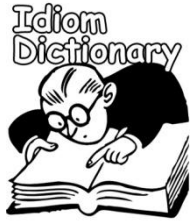


- Let's consider such a class:

```
public final class FinalValue<T> {

    private final T value;

    public FinalValue(T instance) {
        value = instance;
    }

    public T value() {
        return value;
    }
}
```

- Such a class can be used for an alternative to double-checked locking, thanks to the special semantics of `final` fields
  - …but it does not perform necessarily better than using a volatile variable
  - …and it is even less explicit and known, use with care!

Using final wrapper



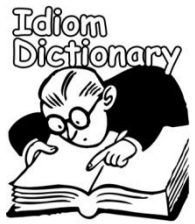
- Using the `FinalValue` class which was sketched on the previous slide:

```
public final class ResourceFactory implements Factory<Resource> {

    private FinalValue<Resource> instance;

    @Override
    public Resource getInstance() {
        FinalValue<Resource> result = instance;

        if (result == null) {
            synchronized (this) {
                if (instance == null) {
                    instance = new FinalValue<>(new MyResource());
                }

                result = instance;
            }
        }

        return result.value();
    }
}
```

- ☠ The local variable `result` is required for the correctness!
- Not a well-established idiom, but a good example for `final` fields

Warning!

Idioms: instance pooling



- Garbage collection has been improved tremendously
  - Allocating and collecting objects is very fast and it can exploit thread confinement and thread locality
  - Understand how the generation-based garbage collection works: the more objects are short-living, the better – pooling can distort the assumptions which the GC is designed for

- Allocating objects is usually cheaper than synchronizing which is required for securing a (shared) pool usually
  - Pooling is sensible for really heavy-weight and expensive objects (like threads, network connections etc.)
  - Pooling is required only in special situations/environments

↳ Rather avoid pooling: it's rarely worth the effort and it is likely to make things worse

Summary

What you should remember

Conclusion

- Monitors: locks, condition variables and predicates
  - Do not forget to unlock, signal, check always the conditions (despite of waking up the thread)
  - Blocking method schema
- Atomic and volatile variables
- Thread interruption support
  - ! Never ignore `InterruptedException` and always try to react on it properly
- Common idioms
  - Common mistakes when implementing double-checked locking and similar patterns
  - The final wrapper trick based on final field semantics

Design concerns

- Designing a thread-safe class
- Extending an implementation
- Implementation techniques

reddot design award

Object fundamentals

- **Object** ~ an entity (real or abstract) that is characterized by its state and behavior

- Only the outer view of an object really matters
  - What the object can do and how to operate with it
  - How the object behaves and reacts

- **Encapsulation ~** hiding internal details of an object
  - Exploits the fact that only the outer view matters

- Concurrency brings additional requirements on encapsulation and on the outer view of an object

- Let's talk about thread-safe classes

What “thread-safe” means

- “A thread-safe class is the class that behaves ‘correctly’ when accessed from multiple threads, regardless of the scheduling (…), and with **no** additional synchronization or other coordination on the part of the calling code”

- The object state is the key
  - Implies outer view of the object
  - Requires keeping some invariants and constraints in order to guarantee post-conditions for the object’s methods

- Mutable object state needs special care in order to reach thread safety

- Immutable objects are always thread-safe (having just one state)
  - Certainly true for truly immutable objects (stateless objects included as being a special case of them)
  - Effectively immutable objects are also thread-safe, but you must prove they are immutable indeed, and they require safe publication

Designing a thread-safe class

- Thread safety can't be ensured without understanding the invariants and post-conditions (which requires understanding of the object's state)
- Constraints on the valid values and state transitions can create atomicity and encapsulation requirements

- Think over the object's **state**
  - Identify the variables that form the object's state
  - Identify the invariants that constrain the state variables
  - Establish a policy for managing concurrent access to the object's state (and document it properly!)

- Think over the object's **state changes**
  - Identify the post-conditions for the operations
    - This gives valid transitions in the state space
    - The transitions imply atomicity requirements

  - Identify state-based preconditions → state-dependent operations can provide a wider range of behavior in a multi-threaded environment (e.g., they may block)
  - Define state ownership: the trouble of defining even the object's state

`+ operation`

Adding a new operation

- Modifying the original class
  - ✓ The safest and easiest way to achieve it
  - ✗ Almost never possible or desirable
- Extending the class
  - ✗ Extending itself is not always easy… and it becomes difficult when considering thread safety
  - • Rather avoid this, unless the class to be extended is not carefully designed and meant for extending
- "Helper" class (and client-side locking)
  - ✗ Even more fragile than extending a class; violates the encapsulation of synchronization policy
- Composition (wrapping and delegating)
  - ✓ The least fragile alternative and also the most common
  - ✗ Sometimes it may bring unexpected problems (e.g., iterators on synchronized collection wrappers)

Thread safety considerations

- Talking about composition… most objects are composed anyway!

- Composition can be used for delegating thread safety – it's a common technique… but must be implemented carefully
  - If a class is composed of multiple **independent** thread-safe state variables and has **no** operations that have any invalid state transitions, then it can delegate thread safety to the underlying state variables
  - If a state variable is thread-safe, does not participate in any invariants that constrain its value, and has no prohibited state transitions for any of its operations, then it can safely be published

Making things simpler

- But after all… consider making things simpler

➢ Encapsulation makes things more manageable
  - Encapsulating data within an object confines access to the data → easier ensuring the data are accessed with the appropriate lock held

➢ Limiting mutability → reducing the state space, the number of transitions, post-conditions etc.

➪ Using immutable objects and composing them removes most of the pain

➪ Removing synchronization costs has often performance benefits

Threads and tasks

- Lifecycle problems
- Executor framework
- Reactive programming

Lifecycle: introduction

- Many entities stand for a service or an activity
  - Encapsulating some action(s) to execute
  - Holding or allocating resources for the actions to be executed

- Needing some means for controlling such entities
  - At least the support for proper terminating an entity and/or its actions is required

- ? Failed/missing lifecycle management results in lost resources, data, poor responsiveness etc.

Lifecycle: JVM support

- JVM provides some lifecycle built-in support
  - Well-defined conditions for JVM termination
  - Support for clean-up actions (although limited)
  - Support for stopping/cancelling actions (thread interruptions)

- JVM built-in support is very low-level
  - Not suitable for direct use, rather a support for implementing higher-level abstractions
  - Some higher-level abstractions are available in libraries (including standard runtime library)
  - Using sensible service frameworks recommended

Shutting down JVM

- A thread may be a *daemon* (see `Thread::isDaemon`)
  - JVM does not terminate (orderly) until all non-daemon threads terminate: daemon threads don't prevent JVM from terminating (and are killed if still running when all "normal" threads finished)
  - Daemon threads are useful for background tasks, especially the tasks that may abort abruptly (usually no I/O is the case though)
    - But they are not a good substitute for proper lifecycle management

- When JVM is shutting down **orderly**, it runs shutdown hooks (see `Runtime::addShutdownHook`) and finalizers
  - There are almost no guarantees about the hooks: their execution order may be arbitrary, they may execute in parallel etc.
  - The hooks must be thread-safe and must finish (rather soon)
  - The finalizers may be executed, but even that is not guaranteed
  - ↳ Avoid finalizers, consider a single shutdown hook… or rather use a sensible service framework with properly defined and supported lifecycle

Cancellation problem: overview

- Apart from JVM termination and using any service framework, cancelling an activity needs often consideration anyway
  - Applications commonly use services that own threads… and the lifetime of those services is usually longer than that of the method that created them
  - ? The thread management is (or should be) encapsulated in the services and **ownership is not transitive** with encapsulation
  - ☠ Improper lifecycle support conflicts with proper JVM termination, leaking or wasted resources etc.

- ➪ Needing a well-defined cancellation protocol
- ➪ Cancellation becomes a design problem!

Cancellation problem: options

- ! Cancellation must be cooperative in the first place – it's a hard requirement (because a thread can't be forced to do anything)

This leaves two basic (low-level) options:

- Cancellation flag/request in a volatile field which is periodically checked

- Using the built-in thread interruption support
  - ! **Interrupting a thread** and **cancelling an activity** are distinct things!
    - The problem of borrowed threads: calling `Thread::interrupt` shall be reserved for the owner of the thread only
    - Higher abstractions should encapsulate the cancellation policy (an example: the timed variant of `Future::get` or `Future::cancel`)

  - × Interruption is just a signal (equal to a flag that is generally checked on thread-blocking operations), sometimes more information is needed
  - × Not everything is interruptible

Cancellation problem: solutions

- Common cancellation techniques:
  - `ExecutorService` (we'll see soon) offers shutdown support → a start for free
  - Using tailored cancellation protocol and synchronization
  - Poison pills

😐 A common termination issue: deciding what terminated

- This might be useful for saving the state for resuming later
- ✘ But difficult: sometimes impossible to avoid "false positives" (a cancelled activity actually finished)

Task-based approach

- A *task* is an abstract, discrete unit of work
  - Tasks should be independent (to be executable in parallel)
  - Each task should represent just a small fraction of the application's processing capacity (for flexibility in scheduling and load balancing)
  - It's good if a task can be encapsulated in an object (brings independence on the executing thread, enables, e.g., SEDA-like architectures or reactive programming style)

- A task does **not** care about execution, but a thread is a too low-level abstraction: an *executor* comes to decouple tasks and execution
  - It's impractical (or even dangerous) to have a thread per a task, so the executive evolved: a thread → a thread pool → an executor
  - An executor can manage the resources according to a policy:
    - How many tasks may execute concurrently, how many may be queued?
    - How to deal with overloading the system?
    - In what order should tasks be executed?
    - In what thread will tasks be executed?

- Java offers a framework based on `Executor` interface

Executor framework

- Avoid using threads directly: rather delegate your tasks to an `Executor` (or `ExecutorService`) instead

- Common thread pools offer much (see `Executors`)
  - ✓ Several policies available – you just have to choose the right one for your intentions (be careful: each choice has consequences)
  - ✓ Support for scheduled tasks (either delayed or periodic): don't use `Timer` anymore, use `ScheduledExecutorService` instead
  - ✓ Support for custom thread creation: `ThreadFactory`
  - ✓ Support for both graceful and abrupt termination
  - ✓ Protection against abnormal thread termination

- More abstractions use cases supported by the framework:
  - ✓ Executing background tasks to wait for later: `CompletionService`
  - ✓ Time-boxing and cancelling tasks: `Future::get`, `Future::cancel`
  - ✓ Result-bearing tasks: `Callable`, `Future` and their descendants
  - ✓ Delayed and periodic tasks

Applying thread pools

- Although the executors decouple task submissions and executions, still threads are underneath and the tasks might be coupled implicitly anyway
  - Dependent tasks: tasks should be independent, but it's not always the case – dependent tasks are acceptable still, but they require a special attention and care then to avoid some hazards (e.g., careful documentation of the requirements)
  - Thread confinement or dependency on a single-threaded environment
  - Using thread locals: violates the task decoupling from the executing thread
  - Response-time sensitivity: long-running tasks can cause problems when not submitted to a properly configured executor as they can block other tasks for too long

☠ Thread starvation deadlock: a task submits another task and waits for its completion – this can cause a deadlock if the thread pool is bounded
  - Actually, any dependency on a bounded resource needs attention

Configuring thread pools
  - **Pool sizing**: it depends on many factors (CPU cores, memory, other resources like connections, nature of the tasks etc.) → guess and profile + make configurable and/or dynamically computed
  - **Queue sizing**: unbounded queues can lead to overload instability → consider carefully and apply possibly a saturation policy

Fork/Join Pool: introduction

- Forking = launching a sub-task to solve a part of a bigger task; the sub-task may be executed asynchronously
- Joining = getting the results of a forked sub-task; it may require waiting for the sub-task to finish

- Fork/Join pattern is meant for independent sub-tasks primarily
  - And also the Fork/Join framework is designed for such usage

- The task implementations are derived from `ForkJoinTask`
  - It is an abstract class implementing `Future` and designed to cooperate with `ForkJoinPool`

- The executor of the tasks is `ForkJoinPool`
  - A special version of `ExecutorService` that is designed specifically to manage the `ForkJoinTask` instances
  - It supports natural forking and joining
  - Maximizes the parallelism transparently with the respect to the configuration, exploiting *work stealing*

Fork/Join Pool: a ~~small~~ example

```java
public final class ListTransformer<U, V>
extends RecursiveTask<List<V>> {

    private final Function<U, V> transformation;
    private final List<? extends U> list;

    public ListTransformer
    (Function<U, V> f, List<? extends U> objects) {
        …
        transformation = f;
        list = objects;
    }

    @Override
    protected List<V> compute() {
        final int size = list.size();
        switch (size) {
        case 0:
            return Collections.emptyList();

        case 1:
            return Collections.<V> singletonList
            (transformation.apply(list.get(0)));

        default:
            final int division = size / 2;

            final RecursiveTask<List<V>> forked
            = new ListTransformer<>(transformation,
            list.subList(division, size));
            forked.fork();

            final RecursiveTask<List<V>> local
            = new ListTransformer<>(transformation,
            list.subList(0, division));

            final List<V> result
            = new ArrayList<>(size);
            result.addAll(local.compute());
            result.addAll(forked.join());
            return result;
        }
    }
}
```

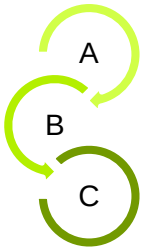
Launch the other half in background

Here we wait for the results of the other half

```java
// Using the task definition above
final Function<int[], Integer> sumFunction
= arg -> IntStream.of(arg).sum();

final List<int[]> matrix = … // Make a matrix
final ForkJoinPool pool = new ForkJoinPool();
final ForkJoinTask<List<Integer>> task
= new ListTransformer<>(sumFunction, matrix);

for (int result : pool.invoke(task)) {
    // Print the sums of the lines… or do whatever else
    System.out.println(result);
}
```

The task(s) are executed here (in parallel if possible)

Reactive programming: about



- "Programming with asynchronous data streams"
  - Oriented around data flows and change propagation, more declarative and lazier than explicit imperative solution
  - An old idea, SEDA might be perceived as one of the predecessors (focusing on data flow)

- Java support available in platform libraries
  - Java 8 provides lambdas + `CompletionStage` with its prominent implementation `CompletableFuture`
  - Java 9 adds `Flow` conforming Reactive Streams specification

- Definitely supersedes direct use of the Fork/Join Pool or many lower-level tools

`CompletableFuture`: overview

- Similar to `Stream` on a level
  - Making a chain of pipeline stages (represented as `CompletionState`) that pass their results down the pipeline
  - Analogous operations (`thenApply` instead of `map`, `thenAccept` instead of `peek` etc.)

- Additional operations due to its asynchronous nature
  - Possibility of branching (e.g., reacting on failures)
  - Possibility of merging existing stages in a new stage

- No terminal operations that makes the pipeline live (usually, made to run from the very start)
- Also a convenient `Future` implementation for stand-alone use

CompletableFuture: example

```
final List<int[]> matrix = … // Make a matrix

// Make a list of futures that compute row sums
final List<CompletableFuture<Integer>> futureSums
= matrix.stream()
    .map(
        row -> CompletableFuture.supplyAsync(
            // This is the sum computation
            () -> IntStream.of(row).sum()
        ))
    .collect(Collectors.toList());

// Make a future that completes when all the row sums
// complete. Unfortunately, allOf needs an array, so
// let's make one from the list
final CompletableFuture<Void> allDone
= CompletableFuture.allOf(futureSums.toArray(
    new CompletableFuture[futureSums.size()])
);

allDone.thenApply(
    // Collect all row sums
    v -> futureSums.stream()
        .map(futureSum -> futureSum.join())
        .collect(Collectors.toList())
.join() // Get the list of the sums
.forEach(System.out::println);
```

Reactive Streams: overview

- An alternative to `CompletableFuture`, but more convenient for different use cases
- Foundation for Publisher/Subscriber model

- Simple enough, but powerful: see `Flow` and its inner interfaces
- Means for subscription control, including backpressure, which allows dynamic switching between push and pull models
- Support for both synchronous and asynchronous models
- Subscribers are invoked in a strictly serial manner in order to keep them simple even within asynchronous streams
- A standard asynchronous publisher implementation ready to use: `SubmissionPublisher`

NIO & NIO.2: I/O evolution

- NIO offered non-blocking I/O based on `SelectableChannel` (for sockets only)
  - See yet `Selector` and `SelectionKey`
  - Employing Reactor pattern
  - Powerful, but difficult to use

- NIO.2 offered asynchronous I/O
  - See `AsynchronousChannel` hierarchy
  - Based on event handlers and/or `Future` – much simpler to use
  - Available for both sockets and files

Summary

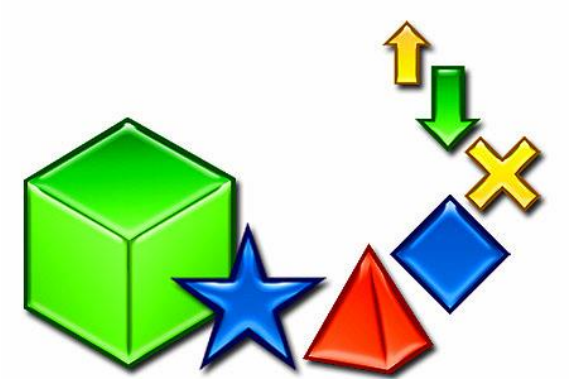
What you should remember

Conclusion

- Always support lifecycle properly
  - Define a proper interruption and cancellation policies
  - Ensure that your code always terminates or does not block JVM termination (checking interruption state or other means)
  - Prefer using high-level abstractions (services and use service frameworks that handle such problems)
- Consider resource management
  - Use bounded thread pools and task queues, define and apply saturation policies
- Employ executor-based abstractions
  - Task running and scheduling, futures – prefer more declarative approach like reactive programming

Building blocks

- Collections and queues
- Synchronizers
- Peeking under the hood



Synchronized collections

- Provided by `Collections::synchronized*`

- Iterators, spliterators and streams returned by such a collection are not concurrent-safe!

- Client-side locking required sometimes (see the documentation to be sure)

- A common problem: hidden iterators
  - Results in `ConcurrentModificationException` (yet in the better case)
  - Difficult to tackle, especially when stacking various wrappers → document thread safety guarantees and requirements carefully always

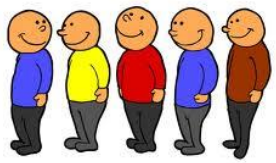
Concurrent collections

- ✓ Improved concurrency (not locking everything always)
- ✓ Concurrent-safe iterators
- 😐 Minor overhead for supporting concurrency

↳ Concurrent collections preferable to synchronized ones in most cases

List	`CopyOnWriteArrayList`	Not efficient for frequent modifications
ConcurrentMap	`ConcurrentHashMap`	
Concurrent NavigableMap	`ConcurrentSkipListMap`	
Set	`CopyOnWriteArraySet`	Good for small sets only
NavigableSet	`ConcurrentSkipListSet`	

Queues: introduction



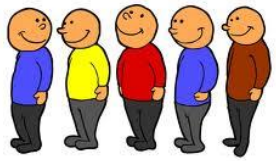
- Root interface: `Queue` (inheriting from `Collection`)
  - Succeeds or fails, no blocking

- **Blocking** queue root interface: `BlockingQueue`
  - Provides additional state-dependent operations which **block** instead of failing (where a normal queue would have failed)
  - Explicitly requires thread safety and establishes *happens-before* relationship between appropriate operations

  ↳ Serial confinement (safe ownership transfers)
  ↳ Exploiting the classic producer-consumer pattern (with workers)
    – The common use of thread pools (managing workers and work items)

- Java Collections contains no sole stack (except for obsolete `Stack`), but provides double-ended queues: `Deque` and `BlockingDeque` (inheriting from the root queue interfaces)

Queues: a reference card



Class	Remarks
`ArrayBlockingQueue`	Performs more predictably than `LinkedBlockingQueue`
`LinkedBlockingQueue`	Performs better than `ArrayBlockingQueue`
`DelayQueue`	Delays item transfer, unbounded only
`PriorityBlockingQueue`	Priority-aware, unbounded only
`LinkedBlockingDeque`	Implements `TransferQueue`, which provides additional methods for direct item transfers; unbounded only
`SynchronousQueue`	Rendezvous semantics, not implementing `TransferQueue` though

Synchronizers: a reference card

Class	Remarks
`AbstractQueuedSynchronizer`	An abstract base class for implementing various synchronizers based on thread queues
`CountDownLatch`	A one-shot gate for delaying the progress of threads
`CyclicBarrier`	A reusable barrier for a fixed number of threads
`Exchanger`	A bidirectional variant of `SynchronousQueue`
`Phaser`	A more general combination of `CountDownLatch` and `CyclicBarrier`
`Semaphore`	A classic synchronizer with the limited number of “permits”

Synchronizers: phasers

- Phasers work in cycles (phases)
  - The phases are numbered, starting with zero
  - It is possible to launch an action (via overriding `Phaser::onAdvance`) when advancing to a new phase
- Varying count of involved parties is possible
  - A party has to *register* for a phase (so there is no need to know the number of the parties in advance)
  - The phase lasts, until all registered parties *arrive* – then the phaser advances to the new phase
  - A registered party may wait for the end of the phase
    - This is how phases synchronize multiple tasks
- Phasers are designed for grouping to trees to achieve high concurrency when a large number of parties is involved

Concurrent troubles

- Liveness hazards
- Performance concerns
- Tips and tricks

Deadlocks

- Lock-ordering deadlocks (the static case)
  - **Cause**: some locks are locked in different order, e.g., synchronized blocks are not ordered properly
  - **Solution**: a fixed global order
- Dynamic lock order deadlocks
  - **Cause**: several locks must be locked like above, but the locks are not determined statically – e.g., the locks are given as parameters
  - **Solution**: ordering the locks before locking dynamically
- Deadlocks between cooperating objects
  - **Cause**: calling another method (typically in another object) with a lock being held
  - **Solution**: being very careful with *alien methods*, using *open calls* whenever possible
- Resource deadlocks
  - **Cause**: a resource is required for a thread to continue (we had an example before: thread starvation deadlock); actually, this is the father of all deadlocks
  - **Solution**: design carefully, careful configuration

Other liveness hazards

- Livelock
  - It occurs when a thread retries an operation which always fails – it is not blocked, but not able to proceed either
  - **Solution**: improve the design (the logic might be flawed), introduce some randomness in the operation
- Starvation
  - It occurs when a thread lacks perpetually a resource needed for its progress (it can be viewed as a kind of the livelock)
  - Generally caused by the lack of fairness → better to resist playing with priorities (e.g., priority inversion problem)
- Poor responsiveness
  - **Solution**: priorities (a rare case when they can be useful), improving lock management, timed operations (to avoid resource consumption on useless actions)

Performance: theory

- Performance has two aspects: “how fast” × “how much”
  - “How much” ~ capacity, throughput
  - “How fast” ~ service time, latency

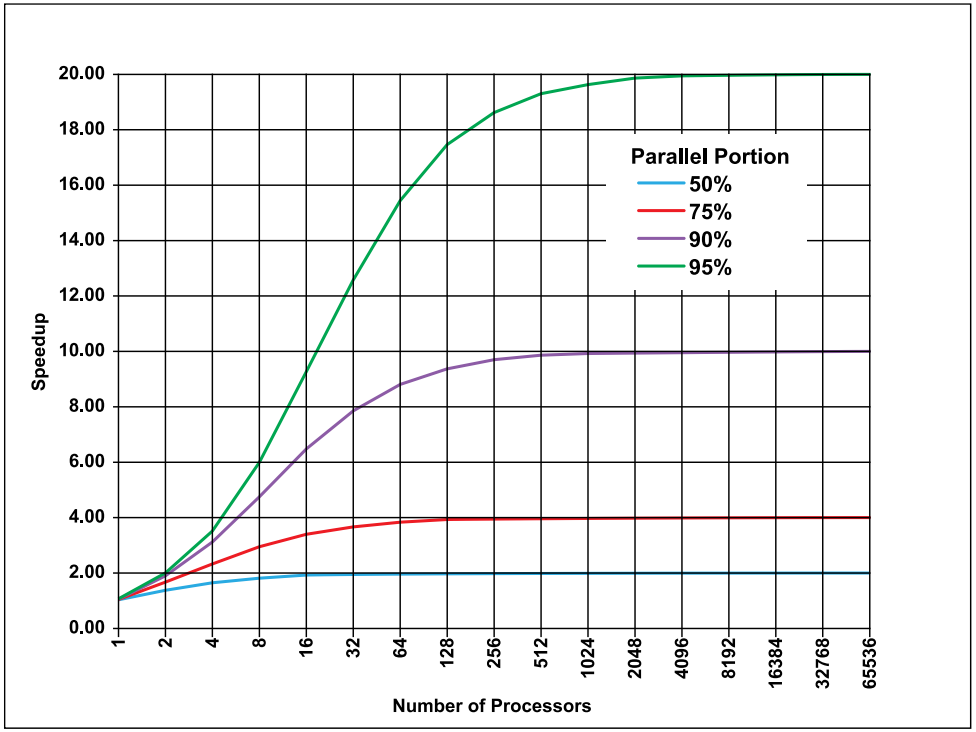
  - ➢ Improving *performance* means doing more work with the same or fewer resources
  - ➢ *Scalability* describes the ability to improve throughput or capacity with adding more resources

- When improving performance, measure and think:
  - ➢ What means “perform better” for your application?
  - ➢ What are the expected data and load – large or small?
  - ➢ What are the hidden costs (including development, maintenance, risks etc.)?

Performance: Amdahl's Law

- Gives the theoretical speedup in latency of the execution of a task at **fixed workload** that can be expected of a system whose resources are improved
- Focusing on bottlenecks

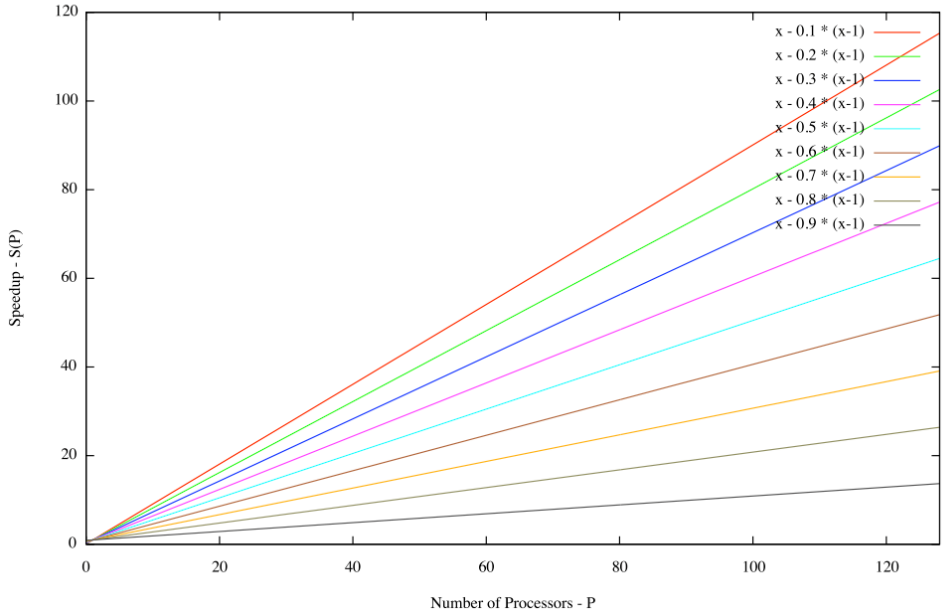
$$S_{\text{latency}}(s) = \frac{1}{(1-p) + \frac{p}{s}}$$



Performance: Gustafson's Law

- Gives the theoretical speedup in latency of the execution of a task at **fixed execution time** that can be expected of a system whose resources are improved
- A more realistic reevaluation of Amdahl's Law

$$S_{\text{latency}}(s) = 1 - p + sp$$



Performance: locking

- Thread mutual exclusion limits concurrency
  - Synchronization is the natural target for optimizations
    - Synchronization limits the space for automatic optimizations
    - JVM and libraries do good work: locking a free lock is cheap; the trouble occurs when the lock is locked and the thread must be blocked → the contention must be reduced!

- How to reduce lock contentions?
  - Narrowing lock scope (“get in, get out”)
  - Changing granularity (lock splitting, lock coarsening)
  - Partitioning locked scope (lock striping)
  - Avoiding hot fields
  - Alternatives to exclusive locks
    - Read-write locks, non-blocking algorithms, avoiding pooling

Performance: other costs

- Context switching
  - Decreasing lock contention helps (since the blocking results usually in a context switch)
  - Try to decrease the number of calls to other routines requiring a context switch, e.g., I/O operations
    - Grouping several operations in an “atomic” one, delegating the operations to a dedicated thread
- Memory synchronization
  - Memory visibility is not entirely free either
    - But still cheaper than locking and it does not block (i.e., it does not limit concurrency so much)

Non-blocking techniques

- Atomic variables can be used as “better volatiles”
  - CAS (compare-and-set) operations are supported by them, and these operations are hardware-enabled usually → it’s light-weight
  - A rule of thumb: the cost of a fast uncontended lock acquisition is approximately twice the cost of a CAS
    - Heavy contention may cause troubles, not the typical one; i.e., CAS-based code is more scalable

- Non-blocking algorithms
  - They use CAS heavily → avoiding to block as much as possible; exploiting cooperation techniques and speculative progress usually
    - Immune to deadlocks and priority inversion, though may tend to livelocks or starvation; after all in general, performing better than blocking alternatives
    - ABA problem (maybe not much for Java)

- A trick useful even for more common use: pairing
  - When two (or even more) values have to changed atomically, make a pair (or tuple) holding them and rather update the container references
    - The trick relies on object reference uniqueness

Thread-local caching

- Sometimes it may be useful to cache some objects on thread-local basis

✓ Avoiding lock contention and pooling
✓ Thread confinement allows using even thread-unsafe objects
✗ Possible problems with object escaping
✗ Possible resource leaks (perhaps should be used for threads within a thread pool under control)

- Java libraries use it quite a lot for frequently used objects like character-set encoders and decoders
- An example of a reasonable use: `ThreadLocalRandom`
  - Each thread has its own isolated generator instance, therefore the contention on the generator instance is avoided
  - Inherits from `Random`: no interface change, just get the instance calling `ThreadLocalRandom::current`

Summary

What you should certainly remember

Overall conclusion

- Think in actions' mutual order and visibility of their effects
- Limit mutability, strive for true immutability
- Make things simpler
- Use locks judiciously
- Use proper abstractions and tools (prefer the higher-level ones if possible)
- Measure, measure, measure

References

- Java language & VM specifications
  https://docs.oracle.com/javase/specs/

- Oracle Java tutorials
  https://docs.oracle.com/javase/tutorial/essential/concurrency/

- Java SE API reference
  https://docs.oracle.com/javase/11/docs/api/

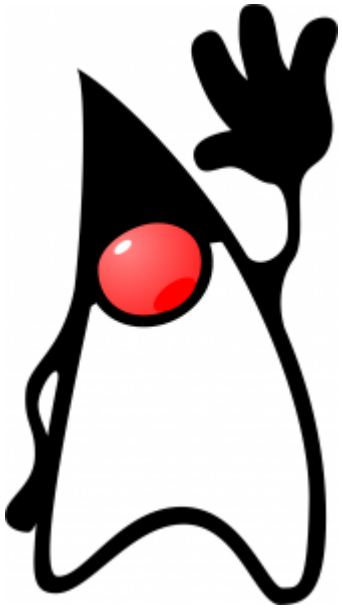
- *Java Concurrency in Practice*
  Brian Goetz

- *Aleksey Shipilëv's blog*
  https://shipilev.net/blog/2014/jmm-pragmatics/
  https://shipilev.net/blog/2016/close-encounters-of-jmm-kind/

- *Doug Lea: Using JDK 9 Memory Order Modes*
  http://gee.cs.oswego.edu/dl/html/j9mm.html

See you soon!

Feel free to contact me in between



Petr Doležal
petr.dolezal@matfyz.cz